Python の関数とデコレータ

スクリプト言語演習第 1 回

本日の内容

- 値・変数・コレクションという Python の基本
- 関数の定義、引数、返り値
- 関数の辞書への登録と、デコレータ

```
def square(x):
    return x * x

f = square          # 変数に代入できる
f(5)                # ==> 25
square.__name__     # ==> 'square'
```

Python の書き方

- 型宣言不要と、別の型の代入
- リストの書き換えと、キーによる辞書の参照
- ブロックの字下げと、4 個のスペース

```
n, name, flag = 42, "Alice", True
x = 10
x = " 文字列"         # 型を持つのは値

xs = [3, 1, 4]
xs.append(9); xs.pop()
d = {"a": 1}
d.get("z", 0)       # 無いキーでも落ちない

[x**2 for x in range(5) if x % 2 == 0]
```

詳細は末尾の付録

値と変数

- 値は整数・文字列・リストなどのオブジェクト
- 変数は値に付ける名前であり、型を固定しない

- type() による値の型の確認

```
x = 10
name = "Alice"
x = " 文字列"        # 型を持つのは値

type(x)              # str
type(name)           # str
```

リストと辞書

- 順番を持つ値の集まりとしてのリスト
- 名前から値を引く対応表としての辞書
- 繰り返し処理で使う基本的なコレクション

```
xs = [3, 1, 4]
xs.append(9)         # [3, 1, 4, 9]

score = {"alice": 80, "bob": 92}
score["bob"]         # 92
score.get("carol", 0)
```

関数の基本

- def による関数の定義
- 引数を受け取り、return で値を返す処理
- 呼び出し時の位置引数と名前付き引数

```
def greet(name, suffix="!"):
    return "Hello, " + name + suffix

greet("Alice")
greet(name="Bob", suffix=".")
```

ディスパッチテーブル

- if の連鎖に伴う関数の肥大化
- 名前による、辞書内の関数の参照
- 演算子追加後も 1 行の calc

```
def calc(op, a, b):
    if op == "add":
```

```
        return a + b
    elif op == "sub":
        return a - b
    else:
        return -1
```

```
import operator
OPS = {"add": operator.add, "sub": operator.sub, "mul": operator.mul}

def calc(op, a, b):
    return OPS.get(op)(a, b)
```

実行時の機能追加

- コンパイル済みの言語での、機能追加時の再コンパイル
- Python では、辞書への関数の追加だけ
- 関数を値として扱うことによる、選択肢の表現

```
OPS["max"] = max          # ここで語彙がひとつ増える
calc("max", 3, 9)         # 9
```

実行中の組み立て直しという、動的言語の特徴

クロージャ

- 関数内での別の関数の定義
- 外側の引数 `n` を保持する返り値の関数
- **クロージャ**という仕組み

```
def multiplier(n):
    def inner(x):
        return x * n      # 外側の n を覚えたまま
    return inner

double = multiplier(2)
double(10)    # 20
```

`@` 記法の糖衣構文

- 関数を受け取り、別の関数を返す `logged`
- `add = logged(add)` と同じ意味
- 定義直後の `f = deco(f)` を表す `@deco`

```
def logged(func):
    def wrapper(*args):
        print(f"{func.__name__}{args}")
        return func(*args)
    return wrapper

def add(a, b):
    return a + b
add = logged(add)
```

```
@logged
def add(a, b):
    return a + b
```

計算結果の再利用

- クロージャが保持する辞書 cache
- タプルをキーにした、引数ごとの結果の保存
- 関数本体を変えない、呼び出しの包装

```
def memo(func):
    cache = {}
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper

@memo
def tarai(x, y, z):
    if x <= y:
        return y
    return tarai(tarai(x-1, y, z),
                 tarai(y-1, z, x),
                 tarai(z-1, x, y))
```

---

1 行で得られる効果

- tarai の本文は変更不要
- 同じ引数に対する再計算の削減
- 1 行の追加による、呼び出し回数の約 5 万分の 1 への削減

tarai(12, 6, 0)	呼び出し回数	時間
@memo なし	12,604,861 回	約 1 秒
@memo あり	**225 回**	一瞬

同じ計算の繰り返しには、一度求めた答えの保存

登録デコレータ

- 関数を辞書に追加して返す command
- @command("shout") による COMMANDS への登録
- 定義と登録の同時実行による、一覧の一貫性

```
COMMANDS = {}

def command(name):
    def deco(func):
        COMMANDS[name] = func
        return func
    return deco

@command("shout")
def shout(text):
    return text.upper() + "!!"
```

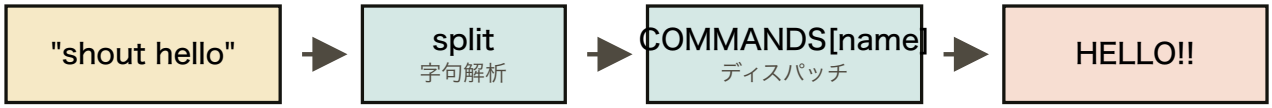
関数の定義位置が、そのままコマンド表

文字列による命令実行

- 先頭の語を命令、残りを引数として解釈
- 辞書にない命令へのエラー
- 文字列から処理系を動かす最小の骨格

```
def run(line):
    name, *args = line.split()
    if name not in COMMANDS:
        raise ValueError(f"知らない命令: {name}")
    return COMMANDS[name](*args)

run("shout hello")    # 'HELLO!!'
```



要点

- **関数は値**。変数・辞書への格納
- **ディスパッチテーブル**。`if` の連鎖を辞書ひとつに集約
- **デコレータ**。中身を変えない機能追加と、`@deco` の `f = deco(f)` への展開

  登録デコレータによる、定義と同時のコマンド表の完成

発展：動的なメソッド生成

- メソッドを 1 つも定義しない `Echo`
- 名前に応じて関数を返す `__getattr__`
- 強力さと、定義を追いにくいという難点

```
class Echo:
    def __getattr__(self, name):
        def method(*args):
            print(f"{name} が {args} で呼ばれました")
        return method

e = Echo()
e.jump(3)
e. どんな名前でも (" いける")
```

読み手には追いにくい仕組み。課題では不要

型・文字列・演算子

- `/` の常に `float` という結果と、`//` の切り捨て
- 整数の桁あふれなしと、処理速度の低下
- 変更不可の文字列と、`s[0] = "X"` のエラー

```
7 / 2       # 3.5
7 // 2      # 3
2 ** 200    # 桁あふれしない
```

```
s = "スクリプト"
s[0], s[-1], s[0:3], s[::-1]
f"{name:>10} | {score:6.2f}"
f"{score = }"                    # print デバッグ用
```

スライスと in による、基本的な文字列処理

制御構造の基本

- 0 < x < 100 のような比較の連鎖
- False と判定される空のコレクション、0、None
- 添字と値を得る enumerate、対応する値を得る zip

```
if 0 < x < 100:
    ...
if not items:
    ...

for i, name in enumerate(names):
for a, b in zip(names, scores):

try:
    n = int(s)
except ValueError:
    n = 0
```

失敗時だけの例外処理

よくある落とし穴

- 元を書き換え、None を返す sort()
- 呼び出し元にも及ぶ、引数のリストの変更
- 定義時に 1 回だけ生成されるデフォルト値の []

```
xs = xs.sort()          # 間違い。sorted(xs) を使う
xs = sorted(xs)

def add(xs, item):
    xs.append(item)    # 呼び出し元のリストが書き換わる
    return xs

def bad(item, xs=[]):
    xs.append(item)    # bad(1) -> [1],  bad(2) -> [1, 2]
```

```
return xs
```

書き換えと新規作成の、明確な使い分け

課題一覧

- 既習の道具を、課題の順に確認
- 前の課題の結果を部品として使う後続の課題
- 任意の発展課題

#	課題	ねらい
1	竹内関数を実装し、呼び出し回数を数える	再帰、爆発する計算量
2	ディスパッチテーブルで電卓	辞書に関数を入れる
3	`@memo` を書き、課題 1 に組み込む	デコレータ、クロージャ
4	`@command` と `run()` でコマンド表	字句解析、ディスパッチ

発展：`@trace` による再帰木の表示 / `__doc__` からの `help` の自動生成

課題の進め方

- 「Colab で開く」から課題を開き、「ドライブにコピーを保存」
- 各課題の `TODO` と、直下の確認セルの `OK`
- 確認セルの保存
  - 教員は別のテストケースで採点するため、削除しても採点結果は不変

課題 3 の `@memo` による、関数本体を変えない呼び出し回数の削減